

Assignment 2

Compilers

1. Consider the following language for regular expressions over the following alphabet $\{a, b\}$:

$$R ::= a \mid b \mid E \mid R \parallel R \mid R \cdot R \mid R^* \mid (R)$$

The meaning of these expressions is not important for this assignment, but E is the empty string (we use E instead of ϵ to avoid confusion with the ϵ symbol used in PDAs), $R_1 \parallel R_2$ is the union of R_1 and R_2 (we use $\parallel$ instead of $\mid$ to avoid confusion with the above $\mid$ BNF symbol), $R_1 \cdot R_2$ is the concatenation of R_1 and R_2 , and R^* is the repetition of R .

- (a) Describe one way in which this grammar is ambiguous and how it could be modified to fix the ambiguity. *(There are multiple ways in which this grammar is ambiguous.)*
- (b) Write a bottom-up parser for the original (non-modified) grammar as a PDA (a pushdown automata). *(The general recipe is provided in `pdas.pdf`. It might help to think about what transitions are shift transitions and what transitions are reduce transitions.)*
- (c) Show that the command $a \parallel b^*$ is accepted by your PDA by providing a sequence of instance descriptions that details all the steps. *(It might again help to think about what steps are shift steps and what steps are reduce steps.)*
- (d) Explain where a shift/reduce ambiguity arises in your computation, and what consequences the decision you have made (shift vs. reduce) has on how the expression is parsed, comparing with what would have happened had you made another decision. *(A shift/reduce ambiguity is when either a shift or a reduce can be performed on the same instance description, while still getting the word to be accepted.)*

[10 marks]

2. Consider the following Intermediate Representation (IR) code:

```
a = a + a
b = a
c = a + 0
a = 2 * a
```

- (a) Translate the above block to assembly code one instruction at a time, using 3 registers, showing how the register and address descriptors are updated, and assuming that a , b , and c initially hold their respective values.
- (b) Optimize the above TAC code, assuming that a , b , and c are used in other blocks, mentioning the names of the optimizations you have used, and showing the code you obtain after each optimization step.
- (c) Describe the cost of the optimized code and compare it to the cost of the non-optimized code (no need to translate the optimized code to assembly too). Which version is the less costly? Justify your answer.

[10 marks]